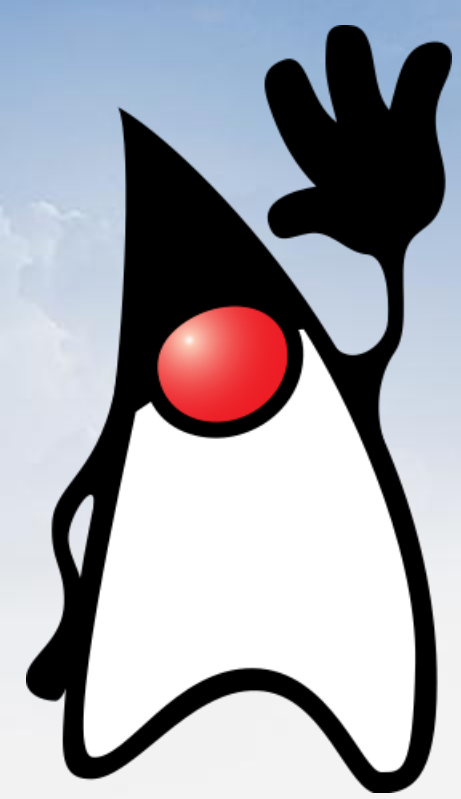




Arrays in Java

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Basic Review of Array

- An array is considered an elementary data structure
- Data structures are collections of related data items
- Arrays are group of variables (called elements) containing values that all have the same type.
- Array size canNOT change once it is initialized.

Name of Array



c[0]

-6

c[1]

2341

c[2]

-53

c[3]

11

c[4]

22

c[5]

34

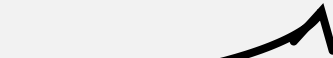
c[6]

34

c[7]

78

Index (subscript)



Java Array Syntax

- Java has different syntax from C when initializing an array.
- In C you learned the following syntax for declaring an array.

- This is wrong! `int x[8];`

- You cannot specify the number of elements in a declaration statement! You will get an error when compiling!

```
int[] arr;  
arr=new int[5];
```

```
int[] arr=new int[3];
```

```
int arr[]; // do not prefer it  
arr=new int[5];
```

```
int arr[]={45,45,223,3424,434};
```

Instantiating the Array

- Once the array is declared, we have to create it!

```
int[] x;
```

```
x = new int[5];
```

- We can also write declaration/instantiation statement in one!

```
int[] arr = new int[5];
```

Array initialization with length

```
int number = 4;
```

```
int[] arr = new int[number];
```

which one is correct?

```
final int NUMBER = 4;
```

```
int[] arr = new int[NUMBER];
```

Both of them are correct!

Common Question

- What is stored in the elements once the array is declared?



We need to test it!
compiler knows the best!

Array Initialization

```
int[] arr={4 ,5 ,6 ,7};  
int[][] arr2d={{2,3},{4,5},{6,7}};
```

```
int[] arr=new int[1];  
arr[0]=4;  
arr[1]=4; //What happens?
```

The code will get compiled fine.
We will get `ArrayIndexOutOfBoundsException` Exception.

Traversing 1D array

```
int[] arr = {4, 5, 6, 7};  
for(int i: arr){  
    System.out.println(i);  
}
```

```
for(int i =0; i< arr.length;i++){  
    System.out.println(arr[i]);  
}
```

2D arrays

- 2D array is a table, 3D is cube, 4D is array of cubes...

	0	1	2	3
0				
1				
2				A
3			B	
4				

where is array[2][3]? A or B?

arr[2][3] => 2nd row 3rd column

arr.length = 5 // number of rows

arr[0].length = 4 , or arr[1].length = 4
number of columns

```
int[][] arr = new int[5][4];
```

Traversing 2D array

```
for(int i =0; i< arr.length;i++){  
    for(int j =0; j< arr[i].length;j++){  
        System.out.print(arr[i][j]+ " ");  
    }  
    System.out.println();  
}
```



if the array row size is same for all, arr[0].length will be same for 1,2,3,4

Traversing 2D array

```
for(int i =0; i< arr.length;i++){  
    for(int j =0 ; j< arr[i].length;j++){  
        System.out.print(arr[j][i]+ " ");  
    }  
    System.out.println();  
}
```



Try changing swapping i and j or for loops as an exercise to see what happens!

variable row size in 2D array

```
//variable row size
int[][] arr2d = {{3}, {4,5}, {4,5,6}};
for(int i=0; i< arr2d.length; i++){
    for(int j=0; j<arr2d[i].length; j++){
        System.out.print(arr2d[i][j]+" ");
    }
    System.out.println();
}
```

```
3
4 5
4 5 6
```

String array

String array Example:

```
String[] arr = new String[8];  
arr[1] = "test1";  
arr[2] = "test2"
```

2DString array Example:

```
String[][] arr = new String[8][2];  
arr[1][1] = "test1";  
arr[2][1] = "test2";
```

The Length Attribute

- In Java, arrays come with a defined attribute called length which holds the number of elements in the array.
- Whenever we pass an array to a function, we do not need to pass the size also. We can access the size of the array inside the function.
- Note that the length attribute does NOT have paranthesis! ~~arr.length()~~

```
public static int[] processArray(int[] arr){  
    for(int i=0; i< arr.length; i++){  
        arr[i] = -1;  
    }  
    return arr;  
}
```

Methods can update the parameter array. Therefore, returning the parameter array is kind of redundant. After the method call, the argument array will be modified already.

Built in Methods for Arrays

- Java has a library methods for Arrays.
- In order to use them, we have to import it from the utility package. Without this import, we will get a compilation error!
 - *import java.util.Arrays;*
- Methods we will observe
 - Arrays.sort()
 - Arrays.toString()
 - Arrays.copyOf()

Arrays Class

- This is the most helpful class
 - Standard C/C++ doesn't have that luxury
- It has some methods which we can use
 - to print the array
 - to sort the array
 - to search the array
 - Initialize an array with value

Arrays.fill()

- You have seen arrays populated manually so far.
- This can be tedious to do.
- What if there was a built in method that populates the array.
- The fill method populates parts of the array
- 4 arguments
 - Array
 - Starting index
 - Ending index
 - Value to populate with

Arrays.fill()

try different parameters for
Arrays.fill()

You may get questions!

```
package main1;  
// Java program to fill a subarray array with limits  
import java.util.Arrays;
```

```
public class Main  
{  
    public static void main(String[] args)  
    {  
        int ar[] = {2, 2, 2, 2, 2, 2, 2, 2, 2};  
  
        // Fill from index 1 to index 4 with 10s  
        Arrays.fill(ar, 1, 5, 10);  
  
        System.out.println(Arrays.toString(ar));  
    }  
}
```

```
package main2;  
// Java program to fill a subarray array  
import java.util.Arrays;
```

```
public class Main  
{  
    public static void main(String[] args)  
    {  
        int ar[] = {2, 2, 1, 8, 3, 2, 2, 4, 2};  
  
        // To fill complete array with a particular value  
        Arrays.fill(ar, 10);  
        System.out.println("Array completely filled" +  
            " with 10\n" + Arrays.toString(ar));  
    }  
}
```

Arrays.sort()

```
int[] array = {45,34,45,67,11,3,99,-2};  
Arrays.sort(array);
```

```
for(int i= 0; i < array.length; i++){  
    System.out.println(array[i]);  
}  
System.out.print(i);// What is the output?
```

```
for(int i: array){  
    System.out.println(i);  
}  
System.out.print(i);// What is the output?
```

String length is length()
array length is length!

by default, sort sorts
the numbers in
ascending order.

Arrays.toString()

This method simply converts data into string format for display.

```
int[] array = {45, 56, 11, 23, 44};  
String convertedOutput = Arrays.toString(array);  
System.out.println(Arrays.toString(array));
```

Arrays.copyOf()

- What if we wanted to copy an array? Or possibly truncate?
- The Array class has a built method that allows us to copy contents of an array.
 - Two Arguments
 - Array
 - An integer that represents new length
 - Can be the same
 - Can be shorter which results in truncating
 - Can be longer which results in padding with default values

Arrays.copyOf()

try different
parameters for
Arrays.copyOf()

You may get
questions!

this is also possible
but longer syntax

empty ones will be
set to 0

Original Array

1 2 3

New array

1 2 3 0 0

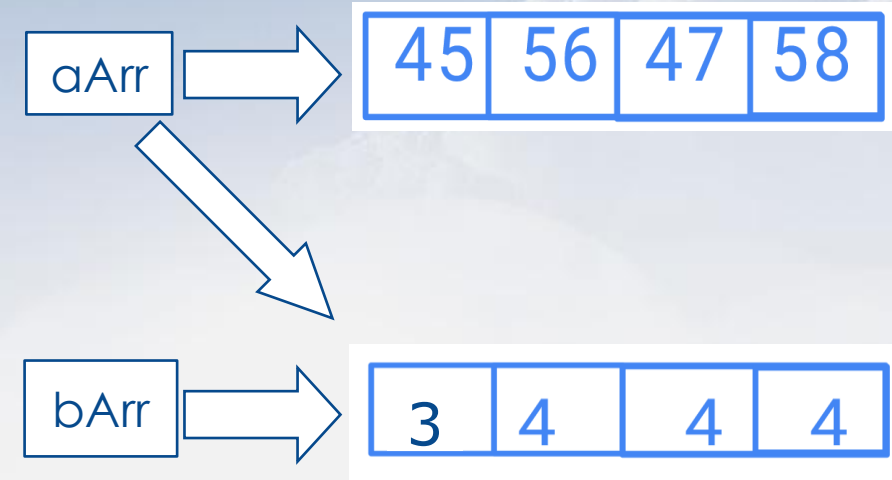
```
package main1;
import java.util.Arrays;
public class Main
{
    public static void main(String args[])
    {
        int[] orig = new int[] {1, 2 ,3};
        System.out.println("Original Array");
        System.out.println(Arrays.toString(orig));
        // copying array org to copy
        int[] copy = Arrays.copyOf(orig, 5);
        // Changing some elements of copy
        //copy[3] = 11;
        //copy[4] = 55;
        System.out.println("New array");
        System.out.print(Arrays.toString(copy));
    }
}
```

Deep copy versus shallow copy

Shallow copy means copy references

```
int[] aArr = {45, 56, 47, 58};  
int[] bArr = {4, 4, 4, 4};  
aArr = bArr; //shallow copied it  
aArr[0] = 3; //changed the original  
System.out.println( Arrays.toString(bArr) );
```

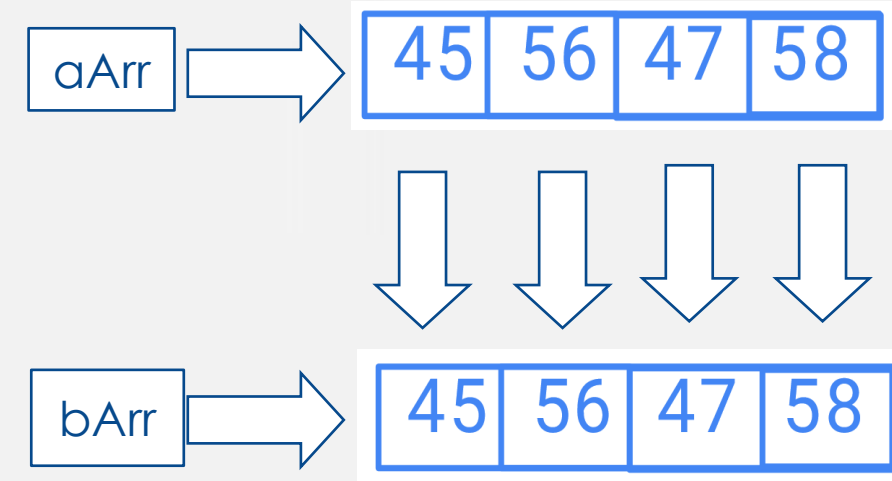
[3, 4, 4, 4]



Deep copy means copy the contents.

```
int[] aArr = {45, 56, 47, 58};  
int[] bArr = {4, 4, 4, 4};  
for(int i=0; i<aArr.length; i++){  
    bArr[i] = aArr[i];  
}  
System.out.println( Arrays.toString(bArr) );
```

[45, 56, 47, 58]



methods with Arrays

shorthand to traverse array is
for (int el : arr){
...
}

Let's see how it works

```
[0, 0, 0, 0, 0]  
[34, 44, 56, 1, 33]
```

When you create an array, initially all the values inside will be zero. The loop inside createFives method did not update anything.

```
package arraysTest;  
import java.util.Arrays;  
public class arrTest{  
    public static void updateSevens(int[] arr){  
        for( int el : arr){  
            el = 7;  
        }  
    }  
    public static int[] createFives(int sizeOfArray){  
        int[] allzeros = new int[sizeOfArray];  
        for( int el : allzeros){  
            el = 5;  
        }  
        return allzeros;  
    }  
  
    public static void main(String[] args){  
        int[] allFives = createFives(5);  
        System.out.println(Arrays.toString(allFives));  
        int[] arr = {34, 44, 56, 1, 33};  
        updateSevens(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```

methods with Arrays

traditional way to traverse array is
for (int i=0; i < arr.length; i++){
... arr[i]
}

Let's see how it works

```
[5, 5, 5, 5, 5]
```

```
[7, 7, 7, 7, 7]
```

```
public static void swap(int[] arr,int pos1,int pos2){  
    int temp=arr[pos1];  
    arr[pos1]=arr[pos2]  
    arr[pos2]=temp;  
}
```

```
package arraysTest;  
import java.util.Arrays;  
public class arrTest{  
    public static void updateSevens(int[] arr){  
        for(int i=0; i < arr.length; i++){  
            arr[i] = 7;  
        }  
    }  
    public static int[] createFives(int sizeOfArray){  
        int[] allzeros = new int[sizeOfArray];  
        for(int i=0; i < allzeros.length; i++){  
            allzeros[i] = 5;  
        }  
        return allzeros;  
    }  
  
    public static void main(String[] args){  
        int[] allFives = createFives(5);  
        System.out.println(Arrays.toString(allFives));  
        int[] arr = {34, 44, 56, 1, 33};  
        updateSevens(arr);  
        System.out.println(Arrays.toString(arr));  
    }  
}
```

So What does those mean?

- DO NOT PREFER the shorthand!
- You will think you are modifying it.
 - You are NOT, You can not edit anything with this form!
- Write with long form if you need to modify the array!
- Know what it does!